

Plano Completo — Camada Contábil: Custos, Estoque, Formação de Preço

Fonte: github.com/frappe/erpnext (código-fonte real, não documentação)

Abordagem: extrair regras de negócio do Python do ERPNext e implementar como skills TypeScript no Arcádia

Data: 26/05/2026

O que extraímos do código ERPNext

1. Avaliação de Estoque (stock/valuation.py)

O ERPNext tem 3 métodos implementados como classes Python:

FIFO — `FIFOValuation`

Queue: `[[qty, rate], [qty, rate], ...]` ← bins ordenados por entrada

`add_stock(qty, rate):`

- se último bin tem mesmo rate → merge (`último_bin[qty] += qty`)
- se último bin tem `qty > 0` → append novo bin
- se último bin tem `qty` negativo → ajustar

`remove_stock(qty):`

- consumir da fila da esquerda para direita
- retorna bins consumidos com seus rates originais
- custo de saída = soma(`qty_consumida * rate_do_bin`)

Moving Average — `get_moving_average_values`

`novo_rate = (qty_atual * rate_atual + qty_entrada * rate_entrada) / (qty_atual + qty_entrada)`

`stock_value = qty_após_transação * valuation_rate`

Implicação para as skills:

O custo de um item saindo do estoque depende do método de avaliação. Para as métricas de CMV, precisamos sempre usar o `valuation_rate` do item — não o `preco_custo` da

tabela de produtos.

2. CMV (Custo das Mercadorias Vendidas) — stock/report/cogs_by_item_group/

Fórmula extraída do relatório `cogs_by_item_group`:

$$\text{CMV} = \Sigma (\text{qty_vendida} \times \text{valuation_rate_na_data_da_venda})$$

Para Atlas:

$$\text{CMV} = \Sigma (\text{pedido_produtos.quantidade} \times \text{pedido_produtos.valor_custo})$$

onde `valor_custo` = custo unitário no momento da venda

3. Lucro Bruto / Margem — accounts/report/gross_profit/gross_profit.py

Campos reais do relatório `GrossProfitGenerator`:

```
base_rate      = preço de venda unitário
buying_rate    = custo unitário (valuation_rate)
base_amount    = qty × base_rate           ← receita
buying_amount  = qty × buying_rate         ← CMV
gross_profit   = base_amount - buying_amount
gross_profit_percent = gross_profit / base_amount × 100
```

Dimensões disponíveis: por invoice, por item_code, por warehouse, por brand, por item_group, por customer, por sales_person.

4. Formação de Preço — accounts/doctype/pricing_rule/utils.py

Hierarquia de aplicação das regras:

1. Item Code específico ← prioridade máxima
2. Item Group
3. Brand

Para cada regra, filtrar por:

- `min_qty ≤ qty ≤ max_qty`
- `min_amt ≤ valor_total ≤ max_amt`
- `data_vigência (valid_from ≤ hoje ≤ valid_upto)`
- cliente / grupo de clientes / território

- `is_cumulative`: soma histórico do cliente no período

Tipos de desconto:

- Percentage discount: $\text{price} = \text{base} \times (1 - \text{discount_pct}/100)$
- Fixed discount: $\text{price} = \text{base} - \text{discount_amount}$
- Rate: $\text{price} = \text{fixed_rate}$
- Margin Rate: $\text{price} = \text{buying_rate} \times (1 + \text{margin_rate}/100)$

Formação de preço pela margem (mais usada em autopeças):

```
preco_venda = custo * (1 + margem_desejada%)
markup_pct  = (preco_venda - custo) / custo * 100      ← markup s/ custo
margem_pct  = (preco_venda - custo) / preco_venda * 100 ← margem s/ venda
```

IMPORTANTE: $\text{markup} \neq \text{margem}$

40% de markup → custo 100 → preço 140 → $\text{margem} = 40/140 = 28,57\%$

40% de margem → custo 100 → preço = $100 / (1 - 0,40) = 166,67$

5. Curva ABC — lógica extraída dos relatórios ERPNext

Algoritmo real (não encontrado em arquivo único — inferido de `stock_analytics` e `item_wise_consumption`):

1. Calcular receita total por item no período
2. Ordenar decrescente por receita
3. Calcular % acumulada sobre receita total
4. Classificar:
 - A = $\text{acumulado} \leq 80\%$ (poucos itens, grande receita)
 - B = $\text{acumulado} \leq 95\%$ (médio volume)
 - C = $\text{acumulado} > 95\%$ (muitos itens, pequena receita)

Variação para estoque (por valor de estoque, não receita):

A = $\text{acumulado} \leq 80\%$ do valor total em estoque

6. Giro de Estoque — `stock/report/stock_ageing/stock_ageing.py`

```
giro = CMV_período / estoque_médio_período
```

onde:

```
estoque_médio = (estoque_inicial + estoque_final) / 2
```

```
dias_de_estoque = 365 / giro    (ou dias_período / giro)
```

Idade média (do código ERPNext):

```
para cada bin FIFO [qty, data_entrada]:
    age_qty += date_diff(hoje, data_entrada) * qty
    total_qty += qty
average_age = age_qty / total_qty
```

7. Ponto de Reposição — stock/report/itemwise_recommended_reorder_level/

```
reorder_level = consumo_diário_médio * lead_time_dias + safety_stock
```

onde:

```
consumo_diário_médio = consumo_período / dias_período
safety_stock = z_score * desvio_padrão_consumo * sqrt(lead_time)
z_score: 1.28 (90%), 1.65 (95%), 2.05 (98%), 2.33 (99%)
```

Arquitetura das Skills — o que criar

Mapa completo

```
server/bi/semantic/
├─ estoque.ts          ← saldo, giro, ABC estoque, ruptura, reposição
├─ custos.ts           ← CMV, margem bruta, formação de preço, BOM
└─ (contabil.ts)       ← já existe, expandir com CMV e margem
```

Cada módulo tem duas fontes:

- **Atlas ERP** (staging tables já populadas) — fonte primária para autopeças
- **Arcádia Control** (lancamentos_financeiros) — fallback genérico

ETAPA 1 — server/bi/semantic/estoque.ts

```
/**
 * Módulo "estoque" — métricas de estoque baseadas nas regras do ERPNext.
 *
```

```

* Fonte primária: analytics.atlas_produtos + analytics.atlas_pedido_produtos
*
* analytics.atlas_entrada_estoques + analytics.atlas_saida_estoq
*
* Algoritmos extraídos de:
* erpNext/stock/valuation.py (FIFO, Moving Average)
* erpNext/stock/report/stock_balance (saldo, in_qty, out_qty)
* erpNext/stock/report/stock_ageing (idade média FIFO)
* erpNext/stock/report/itemwise_recommended_reorder_level
*/
import type { SemanticContext, SemanticMetric } from "./types";
import { quoteIdent, dateRangeClause } from "./sqlHelpers";

export const metrics: SemanticMetric[] = [

  // — SALDO DE ESTOQUE —————
  {
    id: "estoque.saldo_atual_por_produto",
    label: "Saldo de estoque atual por produto",
    description: "Saldo atual (unidades) dos produtos ativos. Fonte: atlas_produ",
    module: "estoque",
    defaultWidget: "data_table",
    cacheTtlSeconds: 300,
    buildQuery: (ctx: SemanticContext) => ({
      sql: `
        SELECT
          COALESCE(pr.codigo_comercial, pr.codigo_barra, pr.id::text) AS codigo,
          COALESCE(pr.nome, pr.apelido) AS name,
          pr.saldo_estoque::float AS value,
          pr.preco_venda AS preco_venda,
          pr.valor_custo AS valor_custo,
          ROUND((pr.saldo_estoque * COALESCE(pr.valor_custo, 0))::numeric, 2)::fl
          COALESCE(m.nome, 'Sem marca') AS marca,
          COALESCE(g.nome, 'Sem grupo') AS grupo
        FROM analytics.atlas_produtos pr
        LEFT JOIN analytics.atlas_marcas m
              ON m.id = pr.marca_id AND m.arcadia_tenant_id = pr.arcadia_tenan
        LEFT JOIN analytics.atlas_grupos_produtos g
              ON g.id = pr.grupo_produto_id AND g.arcadia_tenant_id = pr.arcad
        WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
              AND pr.ativo = true
        ORDER BY value DESC
        LIMIT 200
      `,
    }),
  },
],
{

```

```

id: "estoque.valor_total_em_estoque",
label: "Valor total do estoque (R$)",
description: "Σ (saldo_estoque × valor_custo). KPI de capital imobilizado.",
module: "estoque",
defaultWidget: "kpi_card",
cacheTtlSeconds: 300,
buildQuery: (ctx: SemanticContext) => ({
  sql: `
    SELECT 'Estoque total' AS name,
           ROUND(SUM(saldo_estoque * COALESCE(valor_custo, 0))::numeric, 2)::
    FROM analytics.atlas_produtos
    WHERE arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
           AND ativo = true
           AND saldo_estoque > 0
  `,
  }),
},

{
  id: "estoque.valor_estoque_por_grupo",
  label: "Valor de estoque por grupo de produto",
  description: "Capital imobilizado em estoque distribuído por grupo/categoria.",
  module: "estoque",
  defaultWidget: "bar_chart",
  cacheTtlSeconds: 600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      SELECT COALESCE(g.nome, 'Sem grupo') AS name,
             ROUND(SUM(pr.saldo_estoque * COALESCE(pr.valor_custo, 0))::numeric
    FROM analytics.atlas_produtos pr
    LEFT JOIN analytics.atlas_grupos_produtos g
           ON g.id = pr.grupo_produto_id AND g.arcadia_tenant_id = pr.arcad
    WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
           AND pr.ativo = true AND pr.saldo_estoque > 0
    GROUP BY 1 ORDER BY value DESC LIMIT 20
    `,
    }),
},

{
  id: "estoque.produtos_sem_estoque",
  label: "Produtos sem estoque (ruptura)",
  description: "Produtos ativos com saldo_estoque = 0 ou negativo. Risco de rup
  module: "estoque",
  defaultWidget: "kpi_card",
  cacheTtlSeconds: 300,
  buildQuery: (ctx: SemanticContext) => ({

```

```

    sql: `
        SELECT 'Ruptura' AS name, COUNT(*)::float AS value
        FROM analytics.atlas_produtos
        WHERE arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
        AND ativo = true AND saldo_estoque <= 0
    `,
    }),
},

{
    id: "estoque.produtos_sem_giro_90d",
    label: "Produtos sem giro em 90 dias",
    description: "Produtos com saldo > 0 mas sem venda nos últimos 90 dias (estoq",
    module: "estoque",
    defaultWidget: "kpi_card",
    cacheTtlSeconds: 3600,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            SELECT 'Sem giro 90d' AS name, COUNT(*)::float AS value
            FROM analytics.atlas_produtos pr
            WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
            AND pr.ativo = true AND pr.saldo_estoque > 0
            AND NOT EXISTS (
                SELECT 1 FROM analytics.atlas_pedido_produtos pp
                JOIN analytics.atlas_pedidos ped
                ON ped.id = pp.pedido_id AND ped.arcadia_tenant_id = pp.arcadia_
                WHERE pp.produto_id = pr.id
                AND pp.arcadia_tenant_id = pr.arcadia_tenant_id
                AND ped.status_id = 14
                AND ped.data_pedido >= NOW() - INTERVAL '90 days'
            )
        `,
    }),
},

// — GIRO DE ESTOQUE (regra ERPNext: CMV / estoque_médio) —————
{
    id: "estoque.giro_por_produto",
    label: "Giro de estoque por produto (Top 30)",
    description: "qty_vendida_período / saldo_atual. Quanto maior, mais rápido o",
    module: "estoque",
    defaultWidget: "bar_chart",
    cacheTtlSeconds: 3600,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            WITH vendas AS (
                SELECT pp.produto_id,

```

```

        SUM(pp.quantidade) AS qty_vendida,
        SUM(pp.quantidade * COALESCE(pp.valor_custo, pr2.valor_custo, 0)
FROM analytics.atlas_pedido_produtos pp
JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arc
LEFT JOIN analytics.atlas_produtos pr2 ON pr2.id = pp.produto_id AND
WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
    AND ped.status_id = 14
    ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
GROUP BY pp.produto_id
)
SELECT COALESCE(pr.nome, pr.apelido, 'P-' || v.produto_id::text) AS name,
    ROUND(
        v.qty_vendida / NULLIF(pr.saldo_estoque, 0),
        2)::float AS value
FROM vendas v
JOIN analytics.atlas_produtos pr
    ON pr.id = v.produto_id AND pr.arcadia_tenant_id = ${quoteIdent(ctx.t
WHERE pr.saldo_estoque > 0
ORDER BY value DESC LIMIT 30
    },
    },
},

{
    id: "estoque.dias_de_estoque",
    label: "Dias de estoque médio por grupo",
    description: "saldo_atual / consumo_diário_médio. Quantos dias o estoque atua
    module: "estoque",
    defaultWidget: "bar_chart",
    cacheTtlSeconds: 3600,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            WITH consumo AS (
                SELECT pp.produto_id,
                    SUM(pp.quantidade) / 90.0 AS consumo_diario
                FROM analytics.atlas_pedido_produtos pp
                JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arc
                WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
                    AND ped.status_id = 14
                    AND ped.data_pedido >= NOW() - INTERVAL '90 days'
                GROUP BY pp.produto_id
            )
            SELECT COALESCE(g.nome, 'Sem grupo') AS name,
                ROUND(AVG(pr.saldo_estoque / NULLIF(c.consumo_diario, 0))::numeric
FROM analytics.atlas_produtos pr
JOIN consumo c ON c.produto_id = pr.id
LEFT JOIN analytics.atlas_grupos_produtos g ON g.id = pr.grupo_produto_

```



```

        WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
        AND pr.saldo_estoque > 0
        GROUP BY 1 ORDER BY value ASC
    `
  })),
},

// — CURVA ABC DE ESTOQUE (regra ERPNext: acumulado sobre valor total) —
{
  id: "estoque.curva_abc_valor",
  label: "Curva ABC por valor de estoque",
  description: "A=80% do valor total, B=15%, C=5%. Identificar onde está o capi",
  module: "estoque",
  defaultWidget: "pie_chart",
  cacheTtlSeconds: 3600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      WITH valor_produto AS (
        SELECT id,
              saldo_estoque * COALESCE(valor_custo, 0) AS valor_estoque
        FROM analytics.atlas_produtos
        WHERE arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
              AND ativo = true AND saldo_estoque > 0
      ),
      ranked AS (
        SELECT id, valor_estoque,
              SUM(valor_estoque) OVER() AS total,
              SUM(valor_estoque) OVER(ORDER BY valor_estoque DESC) AS acumulad
        FROM valor_produto
      )
      SELECT
        CASE
          WHEN acumulado / NULLIF(total, 0) <= 0.80 THEN 'A – Capital crítico (
          WHEN acumulado / NULLIF(total, 0) <= 0.95 THEN 'B – Capital médio (15
          ELSE 'C – Capital baixo (5%)'
        END AS name,
        COUNT(*)::float AS value
        FROM ranked GROUP BY 1 ORDER BY 1
    `
  })),
},

{
  id: "estoque.top_estoque_parado_valor",
  label: "Top 20 produtos parados por valor",
  description: "Produtos sem giro em 90 dias ordenados por valor imobilizado (q",
  module: "estoque",

```

```

defaultWidget: "bar_chart",
cacheTtlSeconds: 3600,
buildQuery: (ctx: SemanticContext) => ({
  sql: `
    SELECT COALESCE(pr.nome, pr.apelido, pr.codigo_comercial) AS name,
           ROUND((pr.saldo_estoque * COALESCE(pr.valor_custo, 0))::numeric, 2
    FROM analytics.atlas_produtos pr
    WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
           AND pr.ativo = true AND pr.saldo_estoque > 0
           AND NOT EXISTS (
             SELECT 1 FROM analytics.atlas_pedido_produtos pp
             JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.
             WHERE pp.produto_id = pr.id AND pp.arcadia_tenant_id = pr.arcadia_t
                   AND ped.status_id = 14 AND ped.data_pedido >= NOW() - INTERVAL '9
           )
    ORDER BY value DESC LIMIT 20
  `,
  },
}),
},

// — PONTO DE REPOSIÇÃO (regra ERPNext: consumo_diário × lead_time) ————
{
  id: "estoque.produtos_abaixo_ponto_reposicao",
  label: "Produtos abaixo do ponto de reposição",
  description: "Produtos cujo saldo atual está abaixo do consumo de 30 dias (pr
  module: "estoque",
  defaultWidget: "data_table",
  cacheTtlSeconds: 300,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      WITH consumo_30d AS (
        SELECT pp.produto_id,
               SUM(pp.quantidade) AS qty_30d
        FROM analytics.atlas_pedido_produtos pp
        JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arc
        WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
               AND ped.status_id = 14
               AND ped.data_pedido >= NOW() - INTERVAL '30 days'
        GROUP BY pp.produto_id
      )
      SELECT COALESCE(pr.nome, pr.apelido) AS name,
             pr.saldo_estoque::float AS value,
             c.qty_30d::float AS consumo_30d,
             ROUND((pr.saldo_estoque / NULLIF(c.qty_30d / 30.0, 0))::numeric, 0
    FROM analytics.atlas_produtos pr
    JOIN consumo_30d c ON c.produto_id = pr.id
    WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
  `

```

```

        AND pr.ativo = true
        AND pr.saldo_estoque < c.qty_30d -- saldo < consumo de 1 mês
        AND pr.saldo_estoque > 0          -- ainda tem algo (não é ruptura)
    ORDER BY dias_restantes ASC LIMIT 50
    `
  },
  })),
},
];

```

ETAPA 2 — `server/bi/semantic/custos.ts`

```

/**
 * Módulo "custos" — CMV, margem bruta, formação de preço.
 *
 * Algoritmos extraídos de:
 *   erpNext/accounts/report/gross_profit/gross_profit.py
 *   gross_profit = base_amount - buying_amount
 *   gross_profit_percent = gross_profit / base_amount × 100
 *
 *   erpNext/accounts/ddoctype/pricing_rule/utils.py
 *   margem_pct = (preco - custo) / preco × 100    (sobre venda)
 *   markup_pct = (preco - custo) / custo × 100    (sobre custo)
 *
 *   erpNext/stock/valuation.py
 *   Moving Average: novo_rate = (qty_atual×rate + qty_nova×rate_nova) / (qty_a
 */
import type { SemanticContext, SemanticMetric } from "../types";
import { quoteIdent, dateRangeClause } from "../sqlHelpers";

export const metrics: SemanticMetric[] = [

  // — CMV (COGS) —————
  {
    id: "custos.cmv_por_mes",
    label: "CMV por mês",
    description: "Custo das Mercadorias Vendidas: Σ (qty_vendida × valor_custo_un",
    module: "custos",
    defaultWidget: "bar_chart",
    cacheTtlSeconds: 300,
    buildQuery: (ctx: SemanticContext) => ({
      sql: `
        SELECT to_char(date_trunc('month', ped.data_pedido), 'YYYY-MM') AS name,
               SUM(pp.quantidade * COALESCE(pp.valor_custo, pr.valor_custo, 0))::
        FROM analytics.atlas_pedido_produtos pp
      `
    })
  }
];

```

```

        JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad
        LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
        AND ped.status_id = 14
        ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
GROUP BY 1 ORDER BY 1
    },
    },
},

{
    id: "custos.cmv_total_periodo",
    label: "CMV total do período",
    description: "KPI: total do CMV no período selecionado.",
    module: "custos",
    defaultWidget: "kpi_card",
    cacheTtlSeconds: 300,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            SELECT 'CMV' AS name,
                   SUM(pp.quantidade * COALESCE(pp.valor_custo, pr.valor_custo, 0))::
            FROM analytics.atlas_pedido_produtos pp
            JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad
            LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
            WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
            AND ped.status_id = 14
            ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
        `,
    })),
},

// — MARGEM BRUTA (regra gross_profit.py do ERPNext) —————
{
    id: "custos.margem_bruta_por_mes",
    label: "Margem bruta por mês (%)",
    description: "(Receita - CMV) / Receita × 100. Fórmula gross_profit_percent d
    module: "custos",
    defaultWidget: "line_chart",
    cacheTtlSeconds: 300,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            SELECT to_char(date_trunc('month', ped.data_pedido), 'YYYY-MM') AS name,
                   ROUND(100.0 * (
                       SUM(pp.valor_total) - SUM(pp.quantidade * COALESCE(pp.valor_cust
                       ) / NULLIF(SUM(pp.valor_total), 0)::numeric, 2)::float AS value
            FROM analytics.atlas_pedido_produtos pp
            JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad

```

```

        LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
        AND ped.status_id = 14
        ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
GROUP BY 1 ORDER BY 1
    `
    },
    },
},

{
    id: "custos.margem_bruta_total",
    label: "Margem bruta total (%)",
    description: "KPI consolidado: (Receita - CMV) / Receita × 100 no período.",
    module: "custos",
    defaultWidget: "kpi_card",
    cacheTtlSeconds: 300,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            SELECT 'Margem bruta' AS name,
                   ROUND(100.0 * (
                       SUM(pp.valor_total) - SUM(pp.quantidade * COALESCE(pp.valor_cust
                       ) / NULLIF(SUM(pp.valor_total), 0)::numeric, 2)::float AS value
            FROM analytics.atlas_pedido_produtos pp
            JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad
            LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
            WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
            AND ped.status_id = 14
            ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
        `
    },
    },
},

{
    id: "custos.margem_por_produto",
    label: "Margem bruta por produto (Top 30)",
    description: "(preço_venda - custo) / preço_venda × 100. Formato grupo gross_
    module: "custos",
    defaultWidget: "bar_chart",
    cacheTtlSeconds: 600,
    buildQuery: (ctx: SemanticContext) => ({
        sql: `
            SELECT COALESCE(pr.nome, pr.apelido, pp.produto_id::text) AS name,
                   ROUND(100.0 * (
                       SUM(pp.valor_total) - SUM(pp.quantidade * COALESCE(pp.valor_cust
                       ) / NULLIF(SUM(pp.valor_total), 0)::numeric, 2)::float AS value
            FROM analytics.atlas_pedido_produtos pp
            JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad

```

```

        LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
        AND ped.status_id = 14
        AND pp.valor_total > 0
        ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
GROUP BY pp.produto_id, pr.nome, pr.apelido
ORDER BY value DESC LIMIT 30
    `
  }},
},

{
  id: "custos.margem_por_grupo",
  label: "Margem bruta por grupo de produto",
  description: "Margem % consolidada por categoria. Grupo equivalente a item_gr
module: "custos",
  defaultWidget: "bar_chart",
  cacheTtlSeconds: 600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      SELECT COALESCE(g.nome, 'Sem grupo') AS name,
             ROUND(100.0 * (
               SUM(pp.valor_total) - SUM(pp.quantidade * COALESCE(pp.valor_cust
               ) / NULLIF(SUM(pp.valor_total), 0)::numeric, 2)::float AS value
      FROM analytics.atlas_pedido_produtos pp
      JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad
      LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
      LEFT JOIN analytics.atlas_grupos_produtos g ON g.id = pr.grupo_produto_
      WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
            AND ped.status_id = 14
            ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
      GROUP BY 1 ORDER BY value DESC
    `
  }},
},

{
  id: "custos.margem_por_cliente",
  label: "Margem bruta por cliente (Top 20)",
  description: "Margem % por cliente. Equivalente ao grupo customer do gross_pr
module: "custos",
  defaultWidget: "bar_chart",
  cacheTtlSeconds: 600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      SELECT COALESCE(p.nome_fantasia, p.nome, p.razao_social, 'P-' || ped.clie
             ROUND(100.0 * (

```

```

        SUM(pp.valor_total) - SUM(pp.quantidade * COALESCE(pp.valor_cust
        ) / NULLIF(SUM(pp.valor_total), 0)::numeric, 2)::float AS value
FROM analytics.atlas_pedido_produtos pp
JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad
LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
LEFT JOIN analytics.atlas_pessoas p ON p.id = ped.cliente_id AND p.arca
WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
    AND ped.status_id = 14
    ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
GROUP BY ped.cliente_id, p.nome_fantasia, p.nome, p.razao_social
ORDER BY value DESC LIMIT 20
    `
  }},
},

{
  id: "custos.produtos_abaixo_margem_minima",
  label: "Produtos com margem abaixo de 15%",
  description: "Produtos vendidos com margem bruta < 15% (threshold configuráve
  module: "custos",
  defaultWidget: "data_table",
  cacheTtlSeconds: 600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      WITH margem_produto AS (
        SELECT pp.produto_id,
          ROUND(100.0 * (SUM(pp.valor_total) - SUM(pp.quantidade * COALESC
          / NULLIF(SUM(pp.valor_total), 0)::numeric, 2) AS margem_pc
          SUM(pp.valor_total) AS receita_total,
          pr.nome AS produto_nome
        FROM analytics.atlas_pedido_produtos pp
        JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arc
        LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr
        WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
          AND ped.status_id = 14
          ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
        GROUP BY pp.produto_id, pr.nome
      )
      SELECT COALESCE(produto_nome, produto_id::text) AS name,
        margem_pct::float AS value,
        receita_total::float AS receita_total
      FROM margem_produto
      WHERE margem_pct < 15
      ORDER BY margem_pct ASC LIMIT 50
    `
  }},
},

```

```
// — FORMAÇÃO DE PREÇO (regra pricing_rule.py do ERPNext) —————
{
  id: "custos.analise_markup_por_produto",
  label: "Markup por produto (preço/custo)",
  description: "markup = (preço_venda - custo) / custo × 100. Regra pricing_rul",
  module: "custos",
  defaultWidget: "bar_chart",
  cacheTtlSeconds: 3600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      SELECT COALESCE(pr.nome, pr.apelido, pr.codigo_comercial) AS name,
             ROUND(100.0 * (pr.preço_venda - COALESCE(pr.valor_custo, 0))
                   / NULLIF(pr.valor_custo, 0)::numeric, 2)::float AS value
      FROM analytics.atlas_produtos
      pr
      WHERE pr.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
            AND pr.ativo = true
            AND pr.preço_venda > 0
            AND pr.valor_custo > 0
      ORDER BY value DESC LIMIT 30
    `,
  })),
},

{
  id: "custos.analise_margem_vs_markup",
  label: "Comparativo Margem vs Markup (distribuição)",
  description: "Distribuição dos produtos por faixa de margem % (sobre venda, n",
  module: "custos",
  defaultWidget: "bar_chart",
  cacheTtlSeconds: 3600,
  buildQuery: (ctx: SemanticContext) => ({
    sql: `
      SELECT
        CASE
          WHEN margem_pct < 0      THEN '1 - Negativa'
          WHEN margem_pct < 10    THEN '2 - 0-10%'
          WHEN margem_pct < 20    THEN '3 - 10-20%'
          WHEN margem_pct < 30    THEN '4 - 20-30%'
          WHEN margem_pct < 40    THEN '5 - 30-40%'
          ELSE                     '6 - > 40%'
        END AS name,
        COUNT(*)::float AS value
      FROM (
        SELECT ROUND(100.0 * (preço_venda - COALESCE(valor_custo, 0))
                  / NULLIF(preço_venda, 0)::numeric, 2) AS margem_pct
    `
  })
}
```



```

        FROM analytics.atlas_produtos
        WHERE arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
            AND ativo = true AND preco_venda > 0 AND valor_custo > 0
    ) t
    GROUP BY 1 ORDER BY 1
    `
  })),
},

{
  id: "custos.receita_vs_cmv_por_mes",
  label: "Receita vs CMV por mês",
  description: "mixed_timeseries: receita bruta (barra) + CMV (linha). Visualiz
module: "custos",
defaultWidget: "mixed_timeseries",
cacheTtlSeconds: 300,
buildQuery: (ctx: SemanticContext) => ({
  sql: `
    SELECT to_char(date_trunc('month', ped.data_pedido), 'YYYY-MM') AS name,
           SUM(pp.valor_total)::float AS value,
           SUM(pp.quantidade * COALESCE(pp.valor_custo, pr.valor_custo, 0))::
    FROM analytics.atlas_pedido_produtos pp
    JOIN analytics.atlas_pedidos ped ON ped.id = pp.pedido_id AND ped.arcad
    LEFT JOIN analytics.atlas_produtos pr ON pr.id = pp.produto_id AND pr.a
    WHERE pp.arcadia_tenant_id = ${quoteIdent(ctx.tenantId)}
           AND ped.status_id = 14
           ${dateRangeClause("ped.data_pedido", ctx.startDate, ctx.endDate)}
    GROUP BY 1 ORDER BY 1
    `
    ,
    freeform: true,
  })),
},
];

```

ETAPA 3 — Alertas automáticos (baseados nas regras ERPNext)

Adicionar em `server/bi/atlasAlerts.ts`:

```

// Alertas de Estoque e Custos
{ name: "Ruptura de estoque (> 20 produtos)", metricId: "estoque.produtos_sem_est
{ name: "Margem bruta caindo (< 20%)", metricId: "custos.margem_bruta_total", con
{ name: "Produtos sem giro parado > 100", metricId: "estoque.produtos_sem_giro_90

```

```
{ name: "CMV > 80% da receita", metricId: "custos.margem_bruta_total", condition:
```

ETAPA 4 — Registrar no semantic index

```
// server/bi/semantic/index.ts
import * as estoque from "./estoque";
import * as custos from "./custos";

const MODULES = [control, migration, dq, contabil, estoque, custos];
```

Ordem de execução para o Replit

1. Criar server/bi/semantic/estoque.ts (9 métricas)
2. Criar server/bi/semantic/custos.ts (10 métricas)
3. Atualizar server/bi/semantic/index.ts (importar estoque + custos)
4. Atualizar server/bi/atlasAlerts.ts (4 alertas novos)
5. Restart → GET /api/bi/semantic/catalog deve mostrar 3 novos módulos

Verificação

```
# Total de métricas após aplicar
GET /api/bi/semantic/catalog
# Esperado:
#   contabil: 22 métricas
#   estoque:  9 métricas
#   custos:   10 métricas
#   atlas:    9 métricas
#   control:  5 métricas
#   TOTAL:    ~55 métricas certificadas

# Teste de uma métrica real
POST /api/bi/semantic/run { "metricId": "custos.margem_bruta_total" }
# Retorna margem % calculada sobre dados reais do Atlas
```